

PROJECT J.A.R.V.I.S

BUILDING THE FUTURE OF AI ASYNC SYSTEMS

EPISODE 4: WAKE WORD DETECTION

Making Jarvis Always Listening - No Buttons Needed!

Series: Episode 4/10

Difficulty: Beginner-Friendly

Cost: ₹0 (Free)

WHAT YOU'LL LEARN

- **Wake Word Concept:** How real-world voice activation systems work on smart devices.
- **Smart Speakers Secret:** How devices like Alexa & Google Home remain 24/7 alert without consuming massive background CPU.
- **OpenWakeWord Library:** Installing, configuring, and executing this entirely offline, lightweight AI engine.
- **Autonomous AI:** Engineering Jarvis to launch automatically using only vocal prompts.
- **Audio Streaming:** Splitting, processing, and normalizing live microphone streams in real-time chunks.

EPISODE OVERVIEW

In Episode 3, we successfully gave Jarvis ears (Microphone) and a voice (Speaker). However, there was a major catch: we had to manually click the **"Run"** button every single time we wanted to issue a command. This setup is completely impractical for a real automated AI assistant!

In this episode, we make Jarvis **AUTONOMOUS**. It will run quietly inside your background process 24/7 and spring to life **ONLY** when you call out the magic phrase: `"Hey Jarvis"`.

💡 SIMPLE ANALOGY (EASY TO UNDERSTAND)

Think of a security guard stationed at your main house gate:

❌ **Old Way (Ep 3):** The guard is completely asleep. You have to physically walk up to him and shake his shoulders to wake him up every single time you want to talk.

✅ **New Way (Ep 4):** The guard is perfectly alert. He sits quietly, filtering room noise, and responds instantly the exact millisecond you call out his name.



STEP 1: INSTALL OPENWAKEWORD

What is OpenWakeWord? It is an open-source, ultra-lightweight library designed to detect specific words out of real-time audio. It is exceptionally fast and runs **100% offline** locally on your machine without demanding an internet connection or external servers.

Run the following command within your VS Code Terminal:

```
pip install openwakeword soundfile sounddevice
```

- **openwakeword**: The core machine learning engine that identifies specific vocal models.
- **sounddevice & soundfile**: Libraries engineered to securely intercept and stream your hardware microphone feeds.



STEP 2: THE AI MODELS

OpenWakeWord automatically ships with tiny, highly optimized pre-trained models (specialized AI brains) that focus on specific key phrases:

-  **hey_jarvis_v0.1** — Finetuned to detect the precise trigger phrase "Hey Jarvis"
-  **alexa_v0.1** — Programmed to listen for the phrase "Alexa"

✨ **Note:** You do not need to download these files manually. The framework will automatically fetch them (~500KB format) the very first time your script runs!

STEP 3: THE COMPLETE CODE

Create a fresh file named `wake_word_jarvis.py` inside your project workspace and paste this exact code block:

```
import openwakeword
from openwakeword.model import Model
import sounddevice as sd
import numpy as np
from faster_whisper import WhisperModel
import pyttsx3

print("🤖 Jarvis AI Agent - Always Listening")
print("⚡ Say 'Hey Jarvis' to activate")

# 1. Load the pre-trained "Hey Jarvis" AI Model
owwModel = Model(wakeword_models=["hey_jarvis_v0.1"])
print("✅ Model loaded successfully!")

# 2. Audio Stream Configurations
samplerate = 16000 # Standard frequency standard for speech AI
chunk_size = 1280 # Processes exactly 80 milliseconds of audio at a time

def audio_callback(indata, frames, time, status):
    if status:
        print(f"⚠ Mic Status: {status}")

    # Flatten 2D stream data to a single-dimensional array
    audio_data = indata.copy().ravel().astype(np.float32)

    # Normalization: Level waves neatly between -1.0 and 1.0
    max_val = np.max(np.abs(audio_data))
    if max_val > 0:
        audio_data = audio_data / max_val

    try:
        # Check if the audio chunk matches our wake phrase
        predictions = owwModel.predict(audio_data)

        for model_name, score in predictions.items():
            if score > 0.1:
                print(f"🎯 {model_name} Score: {score:.3f}")
```

```

# If AI confidence exceeds the 0.3 threshold, activate Jarvis
if score > 0.3:
    print("\n✅ WAKE WORD DETECTED! Welcome back.")
    print("🎧 Listening for your command...")

    owwModel.reset() # Prevent feedback loop double-triggers

    # Capture user command window for 3 continuous seconds
    command = sd.rec(int(3 * samplerate),
                     samplerate=samplerate,
                     channels=1,
                     dtype='int16')

    sd.wait()

    audio_float = command.flatten().astype(np.float32) / 32768.0

    # Speech to Text via Faster-Whisper
    model = WhisperModel("tiny", device="cpu", compute_type="int8")
    segments, info = model.transcribe(audio_float, beam_size=1)
    text = "".join([segment.text for segment in segments])

    print(f"📝 You said: {text.strip()}")

    # Synthesizing Output response
    if text.strip():
        print("🗣️ Jarvis responding...")
        engine = pyttsx3.init()
        engine.setProperty('rate', 150)
        engine.say(f"You said: {text}")
        engine.runAndWait()

    print("\n🎧 Listening for 'Hey Jarvis'...")

except Exception as e:
    pass

# 3. Initialize background hardware audio stream
print("\n⌚ Starting audio stream...")
try:
    with sd.InputStream(samplerate=samplerate,
                       channels=1,
                       blocksize=chunk_size,

```

```
        dtype='float32',
        callback=audio_callback):
    print("✅ READY! Speak 'Hey Jarvis' into your microphone.")
    print("📊 Analyzing background audio...\n")

    while True:
        sd.sleep(1000)

except KeyboardInterrupt:
    print("\n\n👋 Jarvis shutting down safely...")
except Exception as e:
    print(f"\n❌ Error: {e}")
```

🔍 CODE EXPLAINED (Simplified Breakdown)

1. Audio Configuration

`samplerate = 16000` forces the system to capture exactly 16,000 computational audio snapshots per single second. This aligns with standard settings for modern speech-to-text models. Meanwhile, `chunk_size = 1280` parses blocks measuring precisely 80 milliseconds. This ensures instant real-time scanning with zero lag.

2. Audio Callback (Background Pipeline)

The `audio_callback` routine loops completely inside a separate asynchronous background processor. Whenever a fresh 80ms sound wave arrives from the hardware microphone, it handles calculations automatically without disturbing or pausing the primary code routine.

3. Wave Normalization

`audio_data = audio_data / max_val` provides software gain control. Different microphones capture varying sound thresholds. This math rescales amplitudes strictly inside a stable `-1.0` to `1.0` boundary, allowing Jarvis to understand you clearly whether you choose to murmur or speak out loud.

4. Threshold Calibration

The AI generates a predictive scale outputting values anywhere from `0.0` (No match) up to `1.0` (Absolute phrase match). Our check limit uses `0.3`. If room accessories or fans cause accident triggers, step this check limit up to `0.4`. If you find it struggles to capture your voice, drop it to `0.2`.

🔧 SYSTEM WORKFLOW

Microphone Stream	Captures constant ambient signals and feeds them down the chain.
SoundDevice Chunks	Slices continuous stream sounds into tiny structured 80ms numerical arrays.
Normalization	Scales arrays to safe machine learning boundaries (-1.0 to 1.0).
OpenWakeWord Test	Computes live probabilities. If calculation score breaks past the 0.3 threshold -> ACTIVATE!
Execution Pipeline	Records user audio context for 3 seconds -> Converts using Whisper STT -> Synthesizes reply via pyttsx3.

▶ STEP 4: HOW TO RUN & TEST

1. Open your terminal environment and launch your engine: `python wake_word_jarvis.py`
2. Move close to your desktop microphone element, clear your throat, and speak out: **"Hey Jarvis"**.
3. Observe the command print:  immediately tracking right across your screen view.
4. Ask any basic prompt quickly (e.g., *"System check status"*). Jarvis will translate your speech into structured text text logs, output them onto the monitor workspace, and reply right back into your speakers!

🎓 KEY RECAP SUMMARY

- **Asynchronous Callbacks:** Running complex peripheral computational loops in background threads without blocking main operational scopes.
- **Edge Machine Learning:** Firing prediction models locally over active inputs without racking up cloud utility billing balances.
- **Data Slicing:** Segmenting data fields down into micro-snapshots to yield lag-free execution loops.

🚀 WHAT'S NEXT? (Episode 5 Preview)

As of right now, Jarvis functions essentially like an interactive recording mimic—it records your vocal spectrum and prints text back matching your original phrase exactly (Echo Routing).

In **Episode 5**, we are going to unpack and embed an active, completely local **Ollama LLM Artificial Brain** right underneath our running wake word setup!

Next Level Setup: You will be able to speak out: *"Hey Jarvis, design an original python automation tool for me,"* and Jarvis will think natively on its feet, construct source logic, and converse back with you 100% offline!

Made with ❤️ by Vamshi VFX • GrowthOS — Building the Future of AI

📺 YouTube: [GrowthOS Telugu](#)